Aprendizaje de Máquina
Sesión 21: Aprendizaje de máquina para textos

Angel Rodríguez

Práctica #6: modificar la representación

Entrega: Martes 20 de Octubre.

Nuevo experimento

Repetir el agrupamiento sustituyendo TF-IDF por otra representación de los documentos.

1. Utilizar `CountVectorizer` para representar frecuencias de términos.
2. Normalizar los vectores mediante norma L_2.
3. Aplicar nuevamente K-means.
4. Comparar la silueta, la curva de inercia y la interpretación de los clusters.

Preguntas finales

¿La representación de los documentos influye en los grupos encontrados?

¿Podríamos utilizar embeddings en lugar de TF-IDF?

Práctica #7: Clasificación de documentos

Entrega: Jueves 22 de Octubre.

Nuevo experimento

Repetir la práctica de clasificación de documentos de la clase anterior (clase #20), sustituyendo TF-IDF por una representación basada en frecuencias de palabras.

1. Cargar los documentos y sus etiquetas.
2. Separar los datos en entrenamiento y prueba.
3. Construir un `Pipeline` con `TfidfVectorizer` y `LogisticRegression`.
4. Entrenar el modelo y evaluar su precisión en el conjunto de prueba.
5. Examinar predicciones correctas e incorrectas.
6. Identificar las palabras con mayor influencia en cada categoría.
7. **Buscar una noticia en Internet** e intentar predecir su categoría utilizando ambos modelos (TF-IDF y CountVectorizer).

Regresión logística

A partir de un vector de características x, calculamos primero

$$z = w^\top x + b.$$

Luego aplicamos la función sigmoide:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

para obtener

$$\boxed{\hat{y} = P(y = 1 \mid x) = \sigma(w^\top x + b).}$$

La salida es una probabilidad

Para clasificación binaria,

$$\hat{y} = P(y = 1 \mid x).$$

Por tanto,

$$P(y = 0 \mid x) = 1 - \hat{y}.$$

Por ejemplo:

$$\hat{y} = 0{,}92$$

significa que el modelo asigna

$$P(\text{positiva} \mid x) = 0{,}92.$$

La regresión logística no produce inicialmente una clase:

produce una probabilidad.

¿Qué aprende el modelo?

Cada característica tiene un peso:

$$w = [w_1, w_2, \ldots, w_p].$$

En clasificación binaria:

$$\boxed{w_j > 0 \Rightarrow \text{evidencia hacia la clase 1}}$$

$$\boxed{w_j < 0 \Rightarrow \text{evidencia hacia la clase 0}}$$

Además,

$$|w_j| \text{ grande}$$

indica una mayor influencia de esa característica.

TF-IDF + Regresión logística

Para un documento d:

$$x_d = [x_1, \ldots, x_p],$$

donde

$$x_j = \text{tf-idf}(t_j, d).$$

La regresión logística calcula

$$z = w_1 x_1 + \cdots + w_p x_p + b.$$

Por tanto, cada palabra contribuye de acuerdo con

importancia en el documento $\times$ peso aprendido

$$x_j w_j.$$

¿Cómo aprendemos los pesos?

Disponemos de un conjunto de entrenamiento

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}.$$

Queremos encontrar

$$w, b$$

que produzcan probabilidades altas para las **clases correctas**.

Para hacerlo necesitamos definir una

función de pérdida.

Entropía cruzada

Para clasificación binaria:

$$\boxed{L_{CE}(\hat{y}, y) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]}$$

Si $y = 1$:

$$L_{CE} = -\log(\hat{y}),$$

mientras que si $y = 0$:

$$L_{CE} = -\log(1 - \hat{y}).$$

Una predicción correcta y segura produce **pérdida pequeña**.

Entrenamiento del modelo

El objetivo es minimizar la pérdida promedio:

$$\boxed{J(\theta) = J(w, b) = \frac{1}{n} \sum_{i=1}^{n} L_{CE}\left(\hat{y}^{(i)}, y^{(i)}\right)}$$

con

$$\hat{y}^{(i)} = \sigma\left(w^{\top} x^{(i)} + b\right).$$

Los parámetros se ajustan mediante métodos de optimización como

descenso de gradiente.

¿Qué ocurre con más de dos clases?

No todos los problemas son binarios.

Por ejemplo:

$$y \in \{\text{positivo}, \text{neutral}, \text{negativo}\}.$$

O:

$$y \in \{\text{deportes}, \text{finanzas}, \text{tecnología}, \text{política}\}.$$

Para $K > 2$ clases utilizamos

regresión logística multinomial

también llamada

softmax regression.

Un conjunto de pesos para cada clase

En clasificación binaria teníamos un vector:

$$w.$$

Para K clases tenemos

$$w_1, w_2, \ldots, w_K.$$

Podemos reunirlos en una matriz:

$$W = \begin{bmatrix} - & - & - & w_1^\top & - & - \\ - & - & - & w_2^\top & - & - \\ & & & \vdots & & \\ - & - & - & w_K^\top & - & - \end{bmatrix}$$

Cada clase recibe su propio score:

$$z_k = w_k^\top x + b_k.$$

Softmax

Los scores

$$z_1, \ldots, z_K$$

se transforman en probabilidades mediante

$$\boxed{P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$$

de modo que

$$0 \leq P(y = k \mid x) \leq 1$$

y

$$\sum_{k=1}^{K} P(y = k \mid x) = 1.$$

Pérdida para varias clases

Representamos la etiqueta correcta como un vector *one-hot*:

$$y = [0, \ldots, 1, \ldots, 0].$$

La salida del modelo es

$$\hat{y} = [\hat{y}_1, \ldots, \hat{y}_K].$$

La entropía cruzada multiclase es

$$\boxed{L_{CE} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)}$$

y, como solo una entrada de y vale 1,

$$L_{CE} = -\log P(\text{clase correcta} \mid x).$$

Dos decisiones diferentes

En clasificación de texto elegimos al menos dos cosas:

Representación

y

Clasificador.

Por ejemplo:

$$\text{CountVectorizer} + \text{LogisticRegression}$$

o

$$\text{TfidfVectorizer} + \text{LogisticRegression}.$$

Cambiar la representación puede modificar sustancialmente el resultado aun cuando **el clasificador sea el mismo**.

Práctica 7: clasificación de documentos

Utilizaremos las cuatro categorías de **20 Newsgroups** de la clase anterior.

1. Cargar los documentos y sus etiquetas.
2. Separar los datos en entrenamiento y prueba.
3. Construir un `Pipeline` con `TfidfVectorizer` y `LogisticRegression`.
4. Entrenar el modelo y evaluar su precisión en el conjunto de prueba.
5. Examinar predicciones correctas e incorrectas.
6. Identificar las palabras con mayor influencia en cada categoría.
7. **Buscar una noticia en Internet** e intentar predecir su categoría utilizando el modelo.

Pregunta

¿Podemos utilizar los textos etiquetados para clasificar automáticamente documentos nuevos?

Discusión: K-means vs. regresión logística

En ambas prácticas utilizamos los mismos documentos y la representación TF-IDF.

- **K-means (no supervisado):** agrupa documentos sin utilizar sus categorías.
- **Regresión logística (supervisado):** aprende a predecir categorías a partir de documentos etiquetados.

Regularización: ¿por qué es necesaria?

Problema: sobreajuste (*overfitting*)

Un modelo puede ajustarse demasiado bien a los datos de entrenamiento, aprendiendo incluso relaciones accidentales o ruido.

- Una característica que aparece únicamente en una clase puede recibir un peso muy grande.
- El modelo puede obtener excelentes resultados en entrenamiento, pero fallar con datos nuevos.
- Queremos un modelo que **generalice**, no que memorice.

Idea de la regularización

Penalizar los coeficientes grandes para evitar que el modelo dependa excesivamente de características particulares del entrenamiento.

Función objetivo regularizada

Sin regularización, buscamos minimizar la función de pérdida (entropía cruzada):

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J_{CE}(\boldsymbol{\theta}),$$

donde

$$J_{CE}(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}).$$

Con regularización

$$\boxed{\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{[J_{CE}(\boldsymbol{\theta}) + \alpha R(\boldsymbol{\theta})]}_{J(\boldsymbol{\theta})}}$$

- $R(\boldsymbol{\theta})$: función de penalización sobre los coeficientes.
- $\alpha \geq 0$: intensidad de regularización.
- Mayor α: mayor penalización de los coeficientes grandes.

Normalmente no se regulariza el término independiente (*bias*).

Regularización L_2 (Ridge)

La regularización L_2 penaliza la suma de los cuadrados de los coeficientes:

$$R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2 = \sum_{j=1}^{n} \theta_j^2$$

La función objetivo se convierte en:

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{\left[J_{CE}(\boldsymbol{\theta}) + \alpha \sum_{j=1}^{n} \theta_j^2 \right]}_{J(\boldsymbol{\theta})}$$

Efecto

- Favorece coeficientes pequeños.
- Generalmente mantiene muchos coeficientes distintos de cero.
- Es relativamente fácil de optimizar:

$$\frac{\partial}{\partial \theta_i} \theta_j^2 = 2\theta_j.$$

Descenso por gradiente con regularización L_2

Recordemos que la función objetivo es:

$$J(\boldsymbol{\theta}) = J_{CE}(\boldsymbol{\theta}) + \alpha \|\boldsymbol{\theta}\|_2^2.$$

Gradiente

Como $\nabla \|\boldsymbol{\theta}\|_2^2 = 2\boldsymbol{\theta}$, tenemos:

$$\boxed{\nabla J(\boldsymbol{\theta}) = \nabla J_{CE}(\boldsymbol{\theta}) + 2\alpha\boldsymbol{\theta}}$$

El paso de descenso por gradiente se modifica a:

$$\begin{aligned}\boldsymbol{\theta}^{(t+1)} &= \boldsymbol{\theta}^{(t)} - \eta \nabla J(\boldsymbol{\theta}^{(t)}) \\ &= \boldsymbol{\theta}^{(t)} - \eta \left[\nabla J_{CE}(\boldsymbol{\theta}^{(t)}) + 2\alpha\boldsymbol{\theta}^{(t)}\right].\end{aligned}$$

$$\boxed{\boldsymbol{\theta}^{(t+1)} = (1 - 2\eta\alpha)\boldsymbol{\theta}^{(t)} - \eta \nabla J_{CE}(\boldsymbol{\theta}^{(t)})}$$

Interpretación: además de minimizar la entropía cruzada, reducimos los coeficientes hacia cero (*weight decay*). El sesgo b no se penaliza.

Regularización L_1 (Lasso)

La regularización L_1 penaliza la suma de los valores absolutos de los coeficientes:

$$\boxed{R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{j=1}^{n} |\theta_j|}$$

La función objetivo se convierte en:

$$\boxed{\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{\left[J_{CE}(\boldsymbol{\theta}) + \alpha \sum_{j=1}^{n} |\theta_j| \right]}_{J(\boldsymbol{\theta})}}$$

Efecto

- Favorece soluciones dispersas: muchos coeficientes exactamente cero.
- Puede actuar como mecanismo de selección de características.
- Es más difícil de optimizar porque $|\theta_j|$ no es derivable en cero.

Descenso por gradiente con regularización L_1

Recordemos la función objetivo:

$$J(\boldsymbol{\theta}) = J_{CE}(\boldsymbol{\theta}) + \alpha \|\boldsymbol{\theta}\|_1.$$

Problema

La función $|\theta_j|$ no es derivable en $\theta_j = 0$. Sin embargo, podemos utilizar un **subgradiente**:

$$s_j = \begin{cases} 1, & \theta_j > 0, \\ -1, & \theta_j < 0, \\ \text{cualquier valor en } [-1, 1], & \theta_j = 0. \end{cases}$$

El paso de descenso por subgradiente es:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \left[\nabla J_{CE}(\boldsymbol{\theta}^{(t)}) + \alpha \mathbf{s}^{(t)} \right]}$$

Interpretación: la penalización L_1 empuja los coeficientes hacia cero. Para obtener soluciones dispersas de forma eficiente suelen utilizarse otros métodos como descenso por coordenadas.

Comparación: regularización L_1 y L_2

Propiedad	L_1	L_2
Penalización	$\sum_j \lvert\theta_j\rvert$	$\sum_j \theta_j^2$
Nombre habitual	Lasso	Ridge
Coeficientes	Muchos ceros	Muchos pequeños
Selección de variables	Sí, implícita	No necesariamente
Optimización	No diferenciable en 0	Diferenciable

En clasificación de textos

- L_1 puede eliminar la influencia de muchas palabras del vocabulario.
- L_2 permite distribuir la influencia entre muchas palabras con pesos pequeños.

Interpretación bayesiana de la regularización

La regularización también puede interpretarse como una preferencia previa sobre los coeficientes.

Regularización L_2

Supone una distribución previa gaussiana centrada en cero:

$$\theta_j \sim \mathcal{N}(0, \sigma^2).$$

$$p(\theta_j) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\theta_j^2}{2\sigma^2}\right).$$

Regularización L_1

Corresponde a una distribución previa de Laplace, también centrada en cero.

Idea: antes de observar los datos, consideramos más plausibles los coeficientes cercanos a cero.

¿Por qué una distribución gaussiana produce L_2?

Con una distribución previa gaussiana, buscamos maximizar (la estimación de **máximo a posteriori (MAP)**):

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \left[\prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}) \prod_{j=1}^{n} p(\theta_j) \right].$$

Al aplicar logaritmos:

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}) + \sum_{j=1}^{n} \log p(\theta_j) \right].$$

Como $\theta_j \sim \mathcal{N}(0, \sigma^2)$, descartando términos constantes:

$$\boxed{\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}) - \frac{1}{2\sigma^2} \sum_{j=1}^{n} \theta_j^2 \right]}$$

Conclusión

Una distribución previa gaussiana equivale a una penalización L_2, con $\alpha = 1/(2\sigma^2)$.

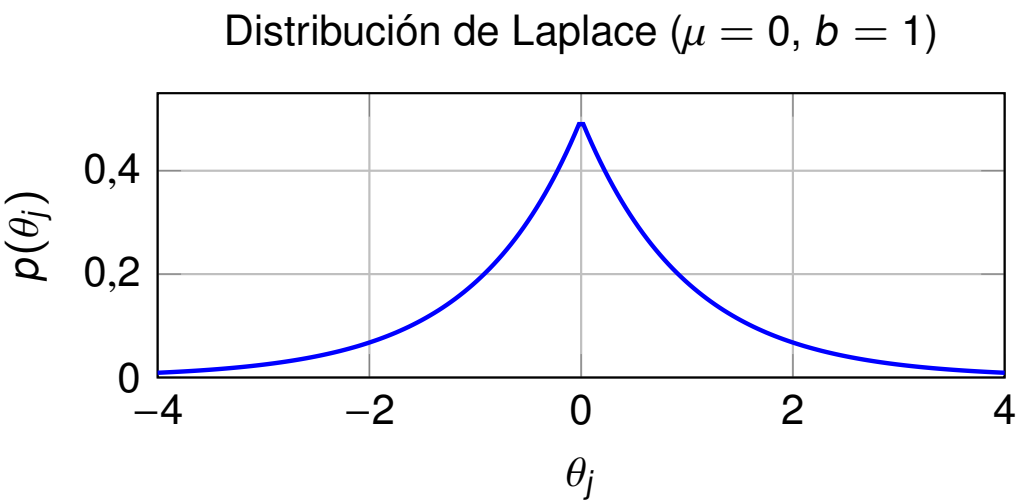
Distribución de Laplace

La función de densidad de una distribución de Laplace es:

$$p(\theta_j \mid \mu, b) = \frac{1}{2b} \exp\left(-\frac{|\theta_j - \mu|}{b}\right)$$

donde:

- μ es el parámetro de localización (centro).
- $b > 0$ es el parámetro de escala (dispersión).



Observación: la densidad es máxima en cero y disminuye exponencialmente conforme aumenta $|\theta_j|$.

¿Por qué una distribución de Laplace produce L_1?

Supongamos una distribución previa de Laplace centrada en cero:

$$p(\theta_j) = \frac{1}{2b} \exp\left(-\frac{|\theta_j|}{b}\right).$$

Aplicando logaritmos:

$$\log p(\theta_j) = \underbrace{-\log(2b)}_{\text{constante}} - \frac{|\theta_j|}{b}.$$

En la estimación MAP, descartando constantes:

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}) - \frac{1}{b} \sum_{j=1}^{n} |\theta_j| \right].$$

Cambiando de maximización a minimización y dividiendo entre m:

$$\boxed{\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left[J_{CE}(\boldsymbol{\theta}) + \alpha \|\boldsymbol{\theta}\|_1 \right]}$$

Conclusión

Una distribución previa de Laplace produce regularización L_1, con $\alpha = \frac{1}{mb}$.

Siguiente paso: ¿qué modelo elegimos?

Podemos construir distintos modelos:

$$\text{CountVectorizer} + \text{LogisticRegression},$$

$$\text{TF-IDF} + \text{LogisticRegression},$$

o incluso modelos basados en embeddings.

Pero:

¿Cómo sabemos cuál generaliza mejor?

Necesitamos utilizar

$$\boxed{\text{métricas} + \text{validación} + \text{comparación de modelos.}}$$

4.5 Evaluación y selección de modelos

Podemos construir distintos modelos para el mismo problema:

$$\text{CountVectorizer} + \text{LogisticRegression}$$

$$\text{TfidfVectorizer} + \text{LogisticRegression}$$

o modelos basados en embeddings.

La pregunta ahora es:

¿Cómo los comparamos correctamente?

Tres preguntas distintas

Para evaluar un modelo necesitamos responder:

1. ¿Qué errores está cometiendo?
2. ¿Qué métrica resume adecuadamente esos errores?
3. ¿Cómo estimamos su desempeño en datos nuevos?

Esto nos lleva a:

$$\boxed{\text{métricas} + \text{validación} + \text{selección de modelos}}$$

Matriz de confusión

Para clasificación binaria comparamos la predicción del sistema con la etiqueta correcta o *gold label*.

		Etiquetas reales		
		Positivo	**Negativo**	
Predicción	**Positivo**	**TP**	**FP**	Precision $= \frac{TP}{TP + FP}$
	Negativo	**FN**	**TN**	
		Recall $= \frac{TP}{TP + FN}$		Accuracy $= \frac{TP + TN}{TP + FP + TN + FN}$

donde:

TP : verdaderos positivos,
FP : falsos positivos,
FN : falsos negativos,
TN : verdaderos negativos.

Accuracy

La exactitud mide la proporción total de clasificaciones correctas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Es una métrica intuitiva, pero puede ser engañosa cuando

las clases están desbalanceadas.

El problema con Accuracy

Supongamos que tenemos

$$1\,000\,000$$

documentos, pero solamente

$$100$$

pertenecen a la clase positiva.

Un clasificador que siempre predice

$$\text{“negativo”}$$

tendría

$$\text{Accuracy} = \frac{999\,900}{1\,000\,000} = 99{,}99\,\%.$$

Precision

Precision responde:

De todo lo que el modelo clasificó como positivo, ¿cuánto era realmente positivo?

$$\boxed{\text{Precision} = \frac{TP}{TP + FP}}$$

Una precisión alta significa

$$\boxed{\text{pocos falsos positivos.}}$$

Recall (sensibilidad)

Recall responde:

De todos los ejemplos realmente positivos, ¿cuántos encontró el modelo?

$$\boxed{\text{Recall} = \frac{TP}{TP + FN}}$$

Un recall alto significa

$$\boxed{\text{pocos falsos negativos.}}$$

Precision vs. Recall

Las dos métricas responden preguntas distintas:

$$\boxed{\text{Precision} = \text{¿qué tan confiables son los positivos detectados?}}$$

$$\boxed{\text{Recall} = \text{¿qué tantos positivos logramos encontrar?}}$$

Dependiendo de la aplicación, un tipo de error puede ser más importante que el otro.

F1-score

Podemos combinar Precision y Recall mediante la media armónica:

$$\boxed{F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}}$$

El valor de F_1 será alto únicamente si

$$\boxed{\text{Precision y Recall son ambas altas.}}$$

Ejemplo

Supongamos que:

$$\text{Precision} = 0{,}90$$

pero

$$\text{Recall} = 0{,}30.$$

Entonces:

$$F_1 = 2\frac{(0{,}90)(0{,}30)}{0{,}90 + 0{,}30} = 0{,}45.$$

Aunque Precision sea alta,

el bajo Recall reduce considerablemente F_1.

¿Qué ocurre con varias clases?

Supongamos una clasificación de correos:

$$y \in \{\text{urgente}, \text{normal}, \text{spam}\}.$$

Podemos calcular Precision y Recall **por separado para cada clase**:

$$P_{\text{urgente}}, \quad P_{\text{normal}}, \quad P_{\text{spam}},$$

$$R_{\text{urgente}}, \quad R_{\text{normal}}, \quad R_{\text{spam}}.$$

Pero también queremos una única métrica global.

Ejemplo: matriz de confusión multiclase

	Categoría real			
Predicción	**Urgente**	**Normal**	**Spam**	**Precisión**
Urgente	8	10	1	$\frac{8}{8+10+1}$
Normal	5	60	50	$\frac{60}{5+60+50}$
Spam	3	30	200	$\frac{200}{3+30+200}$
Exhaustividad	$\frac{8}{8+5+3}$	$\frac{60}{10+60+30}$	$\frac{200}{1+50+200}$	

- **Azul:** predicciones correctas (diagonal).
- **Rojo:** predicciones incorrectas.

Macro-average

En **macro-averaging** calculamos primero la métrica para cada clase y después promediamos.
Por ejemplo:

$$P_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} P_k$$

y de forma análoga para Recall y F_1.

Cada clase contribuye

con el mismo peso.

Micro-average

En **micro-averaging** primero agregamos las decisiones de todas las clases:

$$TP_{\text{total}} = \sum_k TP_k,$$

$$FP_{\text{total}} = \sum_k FP_k.$$

Después calculamos

$$\boxed{P_{\text{micro}} = \frac{TP_{\text{total}}}{TP_{\text{total}} + FP_{\text{total}}}}$$

Las clases frecuentes tienen mayor influencia en el resultado.

Precisión macro y micro en clasificación multiclase

Clase 1: Urgente

	Real +	Real –
Pred. +	8	11
Pred. –	8	340

Clase 2: Normal

	Real +	Real –
Pred. +	60	55
Pred. –	40	212

Clase 3: Spam

	Real +	Real –
Pred. +	200	33
Pred. –	51	83

Global (pooled)

	Real +	Real –
Pred. +	268	99
Pred. –	99	635

$$P_{\text{urgente}} = \frac{8}{8+11} \approx 0{,}42, \qquad P_{\text{normal}} = \frac{60}{60+55} \approx 0{,}52,$$

$$P_{\text{spam}} = \frac{200}{200+33} \approx 0{,}86.$$

Promedios de precisión

$$P_{\text{macro}} = \frac{P_{\text{urgente}} + P_{\text{normal}} + P_{\text{spam}}}{3} \approx 0{,}60$$

$$P_{\text{micro}} = \frac{TP_{\text{total}}}{TP_{\text{total}} + FP_{\text{total}}} = \frac{268}{268+99} \approx 0{,}73$$

Micro vs. Macro

	Micro	**Macro**
Primero	Agrupa ejemplos	Evalúa cada clase
Después	Calcula métrica	Promedia métricas
Peso	Más a clases grandes	Igual por clase

Cuando todas las clases son igualmente importantes,

Macro-average

permite observar mejor el desempeño sobre las clases pequeñas.

Evaluar no es lo mismo que seleccionar

Durante el desarrollo hacemos muchas decisiones:

- Bag of Words o TF-IDF;
- parámetros del vectorizador;
- valor de regularización;
- algoritmo;
- hiperparámetros.

Si utilizamos el conjunto de prueba para tomar estas decisiones,

estamos indirectamente entrenando sobre el test set.

Training, Development y Test

Una división habitual es:

Training set	Development set	Test set

- **Training:** aprender los parámetros del modelo.
- **Development** / **Validation:** seleccionar configuraciones e hiperparámetros.
- **Test:** estimar el desempeño final sobre datos no vistos.

El conjunto de prueba

El test set debe permanecer

sin utilizar durante la selección.

Después de elegir el modelo final:

entrenamos → seleccionamos → evaluamos una sola vez en test

Esto permite obtener una estimación más honesta de la generalización.

Validación cruzada

Cuando el conjunto de datos es limitado, podemos utilizar K-fold cross-validation.

Dividimos los datos en

$$K$$

subconjuntos o *folds*.
En cada iteración:

$$\boxed{K - 1 \text{ folds para entrenamiento} + 1 \text{ fold para validación}}$$

Repetimos el procedimiento K veces.

Ejemplo: 5-fold cross-validation

Con $K = 5$:

Val	Train	Train	Train	Train
Train	Val	Train	Train	Train
Train	Train	Val	Train	Train
Train	Train	Train	Val	Train
Train	Train	Train	Train	Val

Obtenemos:

$$M_1, M_2, \ldots, M_5.$$

Y reportamos

$$\boxed{\bar{M} = \frac{1}{5} \sum_{i=1}^{5} M_i.}$$

⚠¡Importante!⚠Cuidado con el *data leakage*

Supongamos que queremos utilizar TF-IDF.

No debemos hacer:

fit TF-IDF con todos los documentos → cross-validation

porque el vocabulario y los pesos IDF ya contienen información de los folds de validación.

El vectorizador debe ajustarse

dentro de cada fold de entrenamiento.

Validación cruzada: ¿qué debe suceder en cada fold?

En validación cruzada con $k = 5$ por ejemplo, utilizamos el 80 % de los datos para entrenamiento y el 20 % para validación.

Paso	**Entrenamiento (80 %)**	**Validación (20 %)**
1. TF-IDF	`fit_transform()`	`transform()`
2. Clasificador	`fit()`	`predict()`
3. Evaluación	—	F1, accuracy, etc.

Primer fold:

$$\underbrace{D_1, D_2, D_3, D_4}_{\substack{\text{Entrenamiento} \\ \text{ajustar TF-IDF}}} \qquad \underbrace{D_5}_{\substack{\text{Validación} \\ \text{solo transformar}}}$$

Segundo fold:

$$\underbrace{D_1, D_2, D_3, D_5}_{\substack{\text{Entrenamiento} \\ \text{ajustar nuevo TF-IDF}}} \qquad \underbrace{D_4}_{\substack{\text{Validación} \\ \text{solo transformar}}}$$

Importante

El vectorizador TF-IDF se ajusta nuevamente en cada fold, sin utilizar los documentos de validación.

Pipeline y validación cruzada

Scikit-Learn permite evaluar todo el proceso:

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("clf", LogisticRegression())
])

scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=5,
    scoring="f1_macro"
)
```

¿Por qué usar un Pipeline?

En cada fold, Scikit-Learn ejecuta:

fit del vectorizador → transformación → fit del clasificador

utilizando únicamente los datos de entrenamiento de ese fold.

Así evitamos que

información de validación → se filtre al entrenamiento.

Comparando representaciones

Podemos comparar, por ejemplo,

$$\texttt{CountVectorizer} + \texttt{LogisticRegression}$$

contra

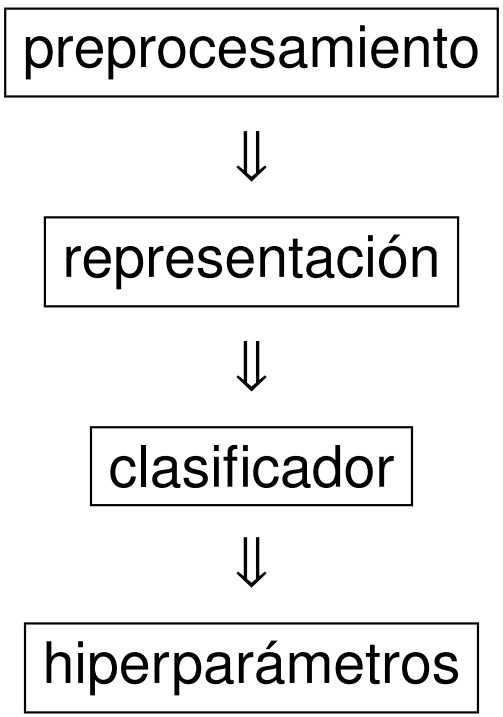
$$\texttt{TfidfVectorizer} + \texttt{LogisticRegression}.$$

La comparación debe realizarse

sobre los mismos folds y con la misma métrica.

¿Qué estamos seleccionando realmente?

Un modelo de texto incluye varias decisiones:



Por tanto, debemos comparar



Selección de hiperparámetros

Por ejemplo, podemos modificar:

- uso de unigramas o bigramas (es decir, considerar dos palabras como “una” sola);
- `min_df`;
- `max_features`;
- tipo de regularización;
- intensidad de regularización.

Cada combinación define un modelo diferente:

$$\theta_{\text{hiper}} \longrightarrow \text{desempeño en validación}$$

Búsqueda de hiperparámetros

Podemos utilizar[1]:

```
from sklearn.model_selection import GridSearchCV

params = {
    "tfidf__ngram_range": [(1,1), (1,2)],
    "clf__C": [0.1, 1, 10]
}

search = GridSearchCV(
    model,
    params,
    cv=5,
    scoring="f1_macro"
)

search.fit(X_train, y_train)
```

[1] Recordemos que podemos ver los parámetros con `print(model.get_params())`

El parámetro C en Scikit-Learn

En `LogisticRegression`, Scikit-Learn utiliza

$$C$$

como el inverso de la fuerza de regularización.
Por tanto:

$$\boxed{C \downarrow \Rightarrow \text{más regularización}}$$

$$\boxed{C \uparrow \Rightarrow \text{menos regularización.}}$$

Por eso podemos incluir C entre los hiperparámetros seleccionados mediante validación cruzada.

¿Una diferencia pequeña es real?

Supongamos que obtenemos:

$$F_1(A) = 0{,}842$$

y

$$F_1(B) = 0{,}847.$$

Tenemos una diferencia

$$\delta = F_1(B) - F_1(A) = 0{,}005.$$

Pero:

¿esa diferencia es estable o podría deberse al azar?

Significancia estadística

Próxima clase...

¡Gracias!